

SKILL WEEK 10

Task 1: Signal Handlers

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

Program

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received. Handling interrupt...\n");

    else if (sig == SIGTERM)
        printf("\nSIGTERM received. Handling termination...\n");
}

int main() {
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    printf("Signal handler program started.\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C to send SIGINT.\n");

    while (1) {
        printf("Process is running...\n");
        sleep(2);
    }

    return 0;
}
```

Output

```
amrutha@blue:~/SKILL_Week10$ nano signal_handler.c
amrutha@blue:~/SKILL_Week10$ gcc signal_handler.c -o signal_handler
amrutha@blue:~/SKILL_Week10$ ./signal_handler
Signal handler program started.
PID: 3624
Press Ctrl+C to send SIGINT.
Process is running...
Process is running...
^C
SIGINT received. Handling interrupt...
Process is running...
Process is running...
Process is running...
^Z
[2]+  Stopped                  ./signal_handler
```

Observation

The program registered handlers for SIGINT and SIGTERM. When Ctrl+C generated SIGINT, the registered handler executed instead of allowing the default action to immediately terminate the process.

Result

Signal handlers were successfully registered and interrupt events were handled.

Task 2: SIGINT Handling

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

To demonstrate how a parent process can remain active while a foreground child process receives SIGINT.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>

pid_t child_pid = -1;

void handle_sigint(int sig) {
    if (child_pid > 0) {
        printf("\nShell received SIGINT. Forwarding to foreground job...\n");
        kill(child_pid, SIGINT);
    } else {
        printf("\nShell protected from SIGINT.\n");
    }
}

int main() {
    signal(SIGINT, handle_sigint);

    printf("Shell started. PID: %d\n", getpid());

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork");
        return 1;
    }

    if (child_pid == 0) {
        printf("Foreground job started. PID: %d\n", getpid());

        while (1) {
            printf("Foreground job running...\n");
            sleep(2);
        }
    }
}
```

```

    printf("Foreground job running with PID %d\n", child_pid);
    printf("Press Ctrl+C to send SIGINT.\n");

    waitpid(child_pid, NULL, 0);

    child_pid = -1;

    printf("Foreground job terminated.\n");
    printf("Job state updated.\n");

    return 0;
}

```

Input

While the program is running:

Ctrl+C

Output

```

amrutha@blue:~/SKILL_Week10$ nano sigint.c
amrutha@blue:~/SKILL_Week10$ gcc sigint.c -o sigint
amrutha@blue:~/SKILL_Week10$ ./sigint
Shell started. PID: 3706
Foreground job running with PID 3707
Press Ctrl+C to send SIGINT.
Foreground job started. PID: 3707
Foreground job running...
Foreground job running...
^C
Shell protected from SIGINT.
Foreground job running...

Shell received SIGINT. Forwarding to foreground job...

Shell protected from SIGINT.
Foreground job running...
Foreground job running...
Foreground job running...
^Z
[3]+  Stopped                  ./sigint

```

Observation

The parent process installed a SIGINT handler. When SIGINT was received, it forwarded the signal to the foreground child process using kill().

Result

SIGINT capture, signal forwarding, and foreground-job termination handling were successfully demonstrated.

Task 3: SIGTSTP / Job Suspension

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

To capture SIGTSTP, suspend a foreground process, preserve its job state, and demonstrate that the process can later be resumed.

Program

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

pid_t child_pid;

void handle_sigstp(int sig) {
    printf("\nSIGTSTP received. Suspending foreground job...\n");

    if (child_pid > 0)
        kill(child_pid, SIGSTOP);
}

int main() {
    signal(SIGTSTP, handle_sigstp);

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork");
        return 1;
    }

    if (child_pid == 0) {
        while (1) {
            printf("Foreground job running...\n");
            sleep(2);
        }
    }

    printf("Foreground job started. PID: %d\n", child_pid);
    printf("Press Ctrl+Z to suspend the job.\n");

    sleep(3);
}
```

```

printf("Sending SIGTSTP to foreground job...\n");
kill(getpid(), SIGTSTP);

sleep(2);

printf("Resuming stopped job using SIGCONT...\n");
kill(child_pid, SIGCONT);

sleep(3);

kill(child_pid, SIGTERM);
waitpid(child_pid, NULL, 0);

printf("Job completed and removed from job table.\n");

return 0;
}

```

Output

```

amrutha@blue:~/SKILL_Week10$ nano sigtstp.c
amrutha@blue:~/SKILL_Week10$ gcc sigtstp.c -o sigtstp
amrutha@blue:~/SKILL_Week10$ ./sigtstp
Foreground job started. PID: 3945
Press Ctrl+Z to suspend the job.
Foreground job running...
Foreground job running...
^Z
SIGTSTP received. Suspending foreground job...
Foreground job running...

SIGTSTP received. Suspending foreground job...
Sending SIGTSTP to foreground job...

SIGTSTP received. Suspending foreground job...
Resuming stopped job using SIGCONT...
Foreground job running...
Foreground job running...
Job completed and removed from job table.

```

Observation

SIGTSTP was used to suspend the foreground job. The process state was preserved and SIGCONT was subsequently used to resume the stopped process.

Result

Job suspension, state preservation, and later recovery were successfully demonstrated.

Task 4: Process Groups

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

To create a process group, assign a process to the group, retrieve the process-group ID, and demonstrate group-based signal handling.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    pid_t pgid;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        setpgid(0, 0);

        pgid = getpgrp();

        printf("Child PID: %d\n", getpid());
        printf("Process Group ID: %d\n", pgid);

        sleep(10);

        printf("Child process exiting.\n");
        exit(0);
    }

    sleep(1);

    setpgid(pid, pid);

    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);
}
```

```
printf("Child Process Group ID: %d\n", getpgid(pid));  
printf("Sending SIGTERM to process group...\n");  
kill(-getpgid(pid), SIGTERM);  
waitpid(pid, NULL, 0);  
printf("Process group execution completed.\n");  
return 0;  
}
```

Input

No keyboard input is required.

The program creates the process group automatically.

Output

```
amrutha@blue:~/SKILL_Week10$ nano process_group.c  
amrutha@blue:~/SKILL_Week10$ gcc process_group.c -o process_group  
amrutha@blue:~/SKILL_Week10$ ./process_group  
Child PID: 3992  
Process Group ID: 3992  
Parent PID: 3991  
Child PID: 3992  
Child Process Group ID: 3992  
Sending SIGTERM to process group...  
Process group execution completed.
```

Observation

A separate process group was created using `setpgid()`. The parent retrieved the child's process-group ID and sent a signal to the process group using a negative process-group identifier.

Result

Process-group creation, group identification, and group-based signal delivery were successfully implemented.